

# Wakacyjne Wyzwanie

Z  KN Solvro

# Jak manipulować danymi?

Karol Stolarczyk  
24.08.2026



KN Solvro

# Rozkład Jazdy



1. Szybka Powtórka
2. Jak wysyłać dane do aplikacji?
3. Przykłady
4. Paginacja
5. Jak to wykorzystać – NestJS
6. Czy nasze dane są poprawne?
7. Jak dokumentować nasze API?
8. Część Praktyczna



KN Solvro

# Szybka powtórka

CRUD I REST API



KN Solvro

# CRUD



**CRUD** czyli **Create, Read, Update, Delete** - cztery podstawowe operacje, które stanowią fundamenty zarządzania danymi w naszych aplikacjach. Pozwalają odpowiednio na **Zapisywanie, Pozyskiwanie, Modyfikowanie i Usuwanie** danych.



KN Solvro

# REST API



- Styl architektury, który bardzo współgra z protokołem HTTP. Używa się go, bo jest prosty, skalowalny i niezależny od technologii. Wszystko w nim traktuje się jak zasób, przez co łatwo przypisać metody HTTP do poszczególnych operacji CRUD.



# Jak to się łączy



## Metoda HTTP

## Operacja CRUD

POST

CREATE

GET

READ

PUT/PATCH

UPDATE

DELETE

DELETE



KN Solvro



# **Jak wysyłać dane do aplikacji?**



KN Solvro

# Wysyłanie danych do aplikacji



Zanim zaczniemy manipulować danymi w naszej aplikacji musimy je jakoś dostarczyć. W REST API informacje o tym, co chcemy zrobić możemy przekazać na trzy główne sposoby, zależnie od ich charakteru i wielkości:

- Path params – do identyfikacji konkretnego zasobu
- Query params – m.in. do filtrowania i sortowania
- Request Body – do wysyłania całkowitych ładunków danych tzw. Payload



# BODY



Najprostsza do zrozumienia forma przesyłu danych to wysłanie aplikacji wiadomości w formacie zrozumiałym przez nią.

- Służy do przesyłania większych złożonych obiektów tzw. Payload
- Standardem w REST API jest format **JSON**

**Uwaga:** najczęściej body zostanie zignorowane dla metod innych niż POST, PUT, PATCH (natomiast nic nie zabrania wysyłania go)



# Path i Query parametry



- Czasami potrzebujemy, a nawet jest prościej, przekazać informacje w ścieżce URI (w linku) ze względu na architekturę REST. Zawierane są w nich ID, albo dodatkowe informacje (metadane), określające co zrobić w danym zapytaniu.



# PRZYKŁADY

<https://api.example.com/users>



KN Solvro

# Part 1. GET:



<https://api.example.com/users>

- Otrzymujemy tablicę użytkowników

```
1  [  
2    {  
3      "id": 1,  
4      "name": "Waldemar Dobrogoszcz"  
5    },  
6    {  
7      "id": 2,  
8      "name": "Igor Grabowski"  
9    }  
10 ]
```

<https://api.example.com/users/1>

- Parametr w ścieżce (Path)
- Wartość „1” w ścieżce reprezentuje ID poszukiwanego rekordu.
- Otrzymaliśmy pojedynczy rekord

```
1  {  
2    "id": 1,  
3    "name": "Waldemar Dobrogoszcz"  
4  }
```



## Part 2. GET:



<https://api.example.com/users?sort=desc&sortBy=id>

- Parametry QUERY: sortujemy użytkowników według id malejąco

```
1  [  
2    {  
3      "id": 2,  
4      "name": "Igor Grabowski"  
5    },  
6    {  
7      "id": 1,  
8      "name": "Waldemar Dobrogoszcz"  
9    }  
10 ]
```



# POST



POST <https://api.example.com/users>

- Jeśli wyślemy na ten endpoint żądanie HTTP z metodą POST oraz dodamy w body:

```
1 { "name": "Gekon Zeropunktowiec" }
```

- To w odpowiedzi otrzymamy:

```
1 { "id": 3, "name": "Gekon Zeropunktowiec" }
```



# **Jak to zrobić na przykładzie Postmana**

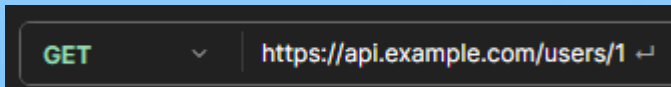


KN Solvro

# Parametry Query i Path



- Możemy dodać je bezpośrednio w ścieżce



- W przypadku Query możemy też dopisywać je w zakładce „Params”

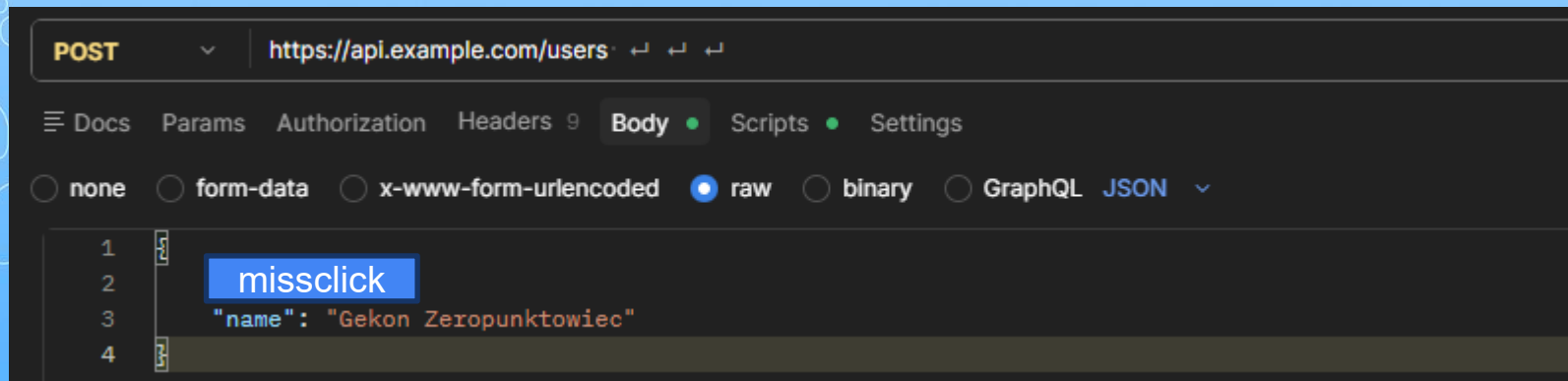
| Key      | Value | Description | Bulk Edit | ... |
|----------|-------|-------------|-----------|-----|
| ✓ sort   | desc  |             |           |     |
| ✓ sortBY | id    |             |           |     |
| Key      | Value | Description |           |     |



# BODY



Żeby wysłać body w formacie JSON



KN Solvro

# Paginacja



KN Solvro

# Przykład z życia

sortowanie

trafność: największa

1 z 100

Najwyżej ocenione  
Przez naszych kupujących

Bestsellery  
Z ostatnich 30 dni

Okazje i promocje

Szybka dostawa

Smart!

Wysyłka z Polski

2026 29" Rower elektryczny ZAZAWORK CF29 250W 48V 15AH MTB Shimano 21 CE PL

5,00 ★★★★★ (5)

27 osób kupiło ostatnio

Rozmiar ramy: L Kolor: czarny Marka: ZAZAWORK

[DODAJ DO PORÓWNIANIA](#)

Sponsorowane

Firma

**2499,00 zł**

darmowa dostawa

166,60 zł x 15 rat [od 0 zł](#) sprawdź

dostawa pojutrze

[DODAJ DO KOSZYKA](#)

ROWER GÓRSKI MĘSKI KOŁA 29 STALOWY MTB 170-195cm 21 BIEGÓW

4,68 ★★★★★ (19)

86 osób kupiło ostatnio

Marka: HURTEX Rozmiar ramy: 21 cali Kolor: czarny

[ZOBACZ 5 OFERT](#) [DODAJ DO PORÓWNIANIA](#)

Promowane

Firma

**889,00 zł SMART**

903,99 zł z dostawą

189,53 zł x 5 rat z [PAY](#) sprawdź

od Super Sprzedawcy

dostawa pojutrze

[DODAJ DO KOSZYKA](#)

ROWER GÓRSKI MTB KOŁA 26 CALI RAMA 17 CALI 21 BIEGÓW AMORTYZATOR 2x TARCZE

4,67 ★★★★★ (12)

77 osób kupiło ostatnio

Marka: MaITrack Rozmiar ramy: 17 cali Kolor: szary

dostępne warianty

[ZOBACZ 10 OFERT](#) [DODAJ DO PORÓWNIANIA](#)

Promowane

Oficjalny sklep

**999,00 zł SMART**

Gwarancja najniższej ceny

1013,99 zł z dostawą

212,98 zł x 5 rat z [PAY](#) sprawdź

od Super Sprzedawcy

dostawa jutro - kup do 16:45

[DODAJ DO KOSZYKA](#)

# Rodzaje paginacji



- **Offset** – pozwala na skakanie po konkretnych stronach, łatwa w implementacji, ale przy ogromnych ilościach danych drastycznie spada jej wydajność. (np. sklepy internetowe)
- **Cursor** – serwer szuka wierszy zaczynając od konkretnego znacznika np. ostatniego id. Ekstremalnie szybka i odporna na duplikaty danych. (np. facebook)

Jak ktoś chce sobie poczytać



KN Solvro



# **Jak to wykorzystać - NestJS**



KN Solvro

# Architektura Nest



- Nest opiera się na architekturze modułowej, czyli logicznie wydzielonych części aplikacji
- Każdy typowy moduł składa się dodatkowo z:
  - **Serwisu** – logika biznesowa i manipulacja danymi
  - **Kontrolera** – definiowanie ścieżek (routing) i przyjmowanie żądań HTTP czy walidacja danych wejściowych
- Plusy tej architektury:
  - **Skalowalność** – pozwala pracować nad różnymi modułami jednocześnie
  - **Utrzymanie** – zmiany w jednym module mają minimalne szanse na uszkodzenie innego
  - **Organizacja** – kod pozostaje zorganizowany i zrozumiały



# Przykładowy Moduł



```
1 import { Module } from '@nestjs/common';
2 import { UserService } from './user.service';
3 import { UserController } from './user.controller';
4
5 @Module({
6   controllers: [UserController],
7   providers: [UserService],
8 })
9 export class UserModule {}
10
```



KN Solvro



# Kontroler



- Opisujemy wszystkie endpointy danego zasobu.
- Za pomocą dekoratorów @Get(), @Post() itd. nad metodami oznaczamy je jako nasze endpointy, które przyjmujemy dane z DTO (o tym później) lub z parametrów omawianych wcześniej.
- Deleguje zadania do serwisów, które zawierają logikę biznesową.



KN Solvro



# Przykładowy Kontroler

```
1 import { Controller, Get, Post, Body, Patch, Param, Delete } from '@nestjs/common';
2 import { UserService } from '../user.service';
3 import { CreateUserDto } from '../dto/create-user.dto';
4 import { UpdateUserDto } from '../dto/update-user.dto';
5
6 @Controller('user')
7 export class UserController {
8   constructor(private readonly userService: UserService) {}
9
10  @Post()
11  create(@Body() createUserDto: CreateUserDto) {
12    return this.userService.create(createUserDto);
13  }
14
15  @Get()
16  findAll() {
17    return this.userService.findAll();
18  }
19
20  @Get('/:id')
21  findOne(@Param('id') id: string) {
22    return this.userService.findOne(+id);
23  }
24
25  @Patch('/:id')
26  update(@Param('id') id: string, @Body() updateUserDto: UpdateUserDto) {
27    return this.userService.update(+id, updateUserDto);
28  }
29
30  @Delete('/:id')
31  remove(@Param('id') id: string) {
32    return this.userService.remove(+id);
33  }
34 }
35
```



KN Solvro

# Serwis



- Znane też jako provider
- Zawierają logikę biznesową
- Serwis może być podpięty pod dedykowany mu kontroler, lub zapewnia ogólną funkcjonalność (serwis dostępu do bazy danych)



KN Solvro

# Przykładowy Serwis



```
1 import { Injectable } from '@nestjs/common';
2 import { CreateUserDto } from '../dto/create-user.dto';
3 import { UpdateUserDto } from '../dto/update-user.dto';
4
5 @Injectable()
6 export class UserService {
7   create(createUserDto: CreateUserDto) {
8     return 'This action adds a new user';
9   }
10
11   findAll() {
12     return 'This action returns all user';
13   }
14
15   findOne(id: number) {
16     return `This action returns a #${id} user`;
17   }
18
19   update(id: number, updateUserDto: UpdateUserDto) {
20     return `This action updates a #${id} user`;
21   }
22
23   remove(id: number) {
24     return `This action removes a #${id} user`;
25   }
26 }
27
```



# DTO



- ang. Data transfer object
- DTO wykorzystujemy do przekazywania danych między warstwami aplikacji (kontroler – serwis)
- Zapewnia nam walidację oraz serializację danych



# Przykładowe DTO



```
1  export class CreateUserDto {  
2      name!: string;  
3  
4      email!: string;  
5  
6      age?: number;  
7  }  
8
```



**Czy nasze dane są poprawne?**



KN Solvro

# class-validator i class-transformer



- class-transformer – odbiera z **BODY** surowy format JSON a następnie transformuje go w obiekt klasy TypeScript
- class-validator – dostaje gotowy obiekt po przetworzeniu i sprawdza jego pola na podstawie dekoratorów (np. @IsString(), @IsEmail() itd.). Jeśli znajdzie błędy, zwraca ich listę, którą NestJS przechwytuje i zmienia odpowiedź na kod 400 (Bad Request)

```
$ npm i --save class-validator class-transformer
```



KN Solvro



# Pipes

- Wbudowany mechanizm NestJS
- Celem jest transformacja i jednocześnie walidacja danych.



```
1 @Get('/:id') //tutaj dostaniemy 500
2 findOne(@Param('id') id: string) {
3   return this.userService.findOne(+id);
4 }
5
6 @Get("/:id") //tutaj dostaniemy 400
7 findOneValidated(@Param('id', ParseIntPipe) id: number) {
8   return this.userService.findOne(id);
9 }
```



# Walidacja DTO



- Bardzo prosta, umieszczamy odpowiednie dekoratory. W przypadku błędu, użytkownik dostanie odpowiedź z kodem 400 i opcjonalną wiadomość, która może być mniej lub bardziej szczegółowa – zależy to tylko od nas.
- Za pomocą wbudowanych walidatorów jesteśmy w stanie sprawdzić większość kryteriów.
- Jeżeli jednak wbudowane walidatory to za mało, możemy tworzyć własne i przypisywać za pomocą `@Validate(NazwaKlasy)`



# Ulepszone DTO



```
1 export class CreateUserDto {
2     @IsString({ message: "Name must be a string" })
3     @NotEmpty({ message: "Name is required" })
4     @MaxLength(50, { message: "Name must be at most 50 characters" })
5     name!: string;
6
7     @IsEmail()
8
9     email!: string;
10
11     @IsNumber({}, { message: "Age must be a number" })
12     @Min(0, { message: "Age must be a positive number" })
13     @Max(150, { message: "Age cannot be greater than 150" })
14     age?: number;
15 }
16
```



# Co zrobić, żeby walidacja działała



Nasze walidatory nie będą działały od razu, mamy dwa sposoby, żeby Nest brał je pod uwagę:

- Przy każdym endpointzie dekorować go:

```
@UsePipes(new ValidationPipe())
```

- Zadeklarować walidacje globalnie w pliku main.ts:

```
app.useGlobalPipes(new ValidationPipe());
```



# Przykładowy Własny Walidator



```
1 import { isString, ValidatorConstraint, ValidatorConstraintInterface } from "class-validator";
2
3 @ValidatorConstraint({ name: "containsA", async: false})
4 export class containsA implements ValidatorConstraintInterface {
5     validate(text: string): boolean {
6         if(!isString(text)) return false;
7         return text.toLowerCase().includes('a');
8     }
9
10    defaultMessage(): string {
11        return 'Text must contains A'
12    }
13 }
```



# **Jak dokumentować nasze API?**



KN Solvro

# Dokumentacja



Aby wygodnie, wydajnie i przyjemnie pracowało się backend i frontend developerom warto dokumentować w jakiś sposób stan endpointów. Rozwiązanie, które zostało przyjęte na ogół to Swagger. Jest to najczęściej plik .yaml/.yml, w ustalonym formacie gdzie zapisywane są informacje o endpointach.



KN Solvro

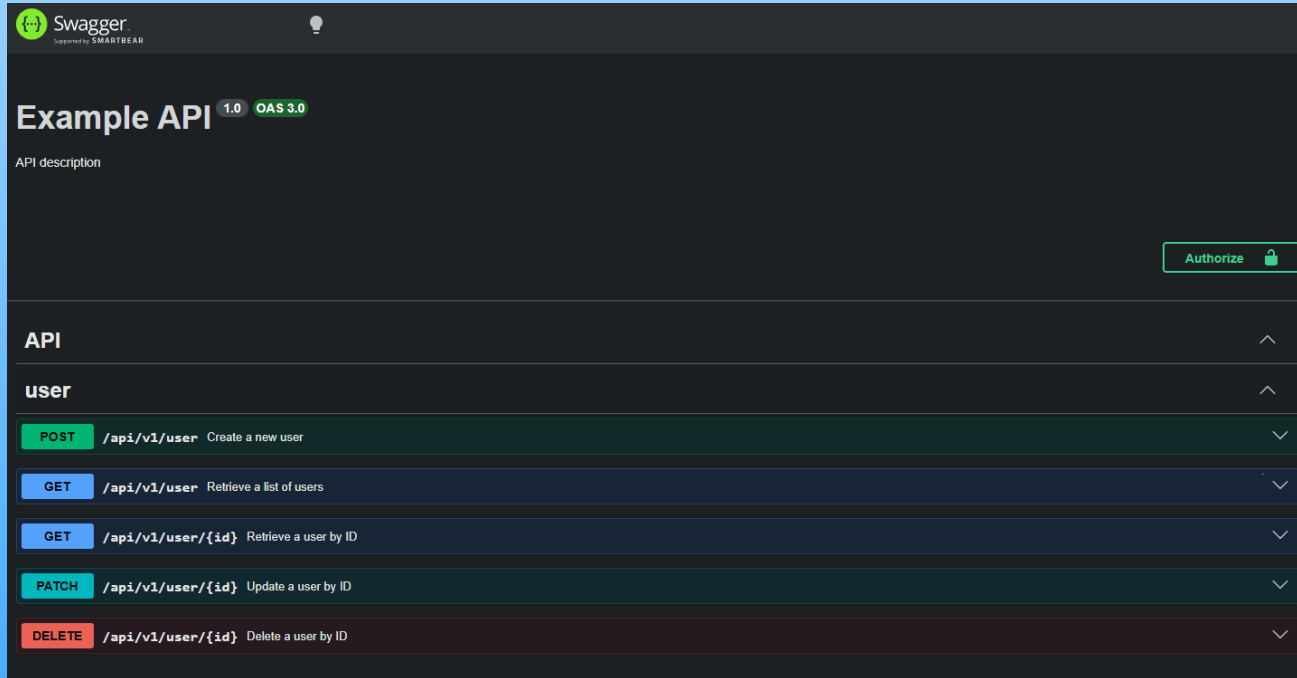
# Swagger UI



- Kiedy już zainicjujemy Swaggera w naszej aplikacji, uzyskamy dostęp do wygenerowanego automatycznie frontendu, który w prosty sposób wyświetli nam wszystkie informacje o endpointach zawartych w aplikacji.
- Naszym zadaniem jest zapisanie w kodzie dodatkowych tagów np. @ApiOperation, @ApiResponse itd., żeby poprawić jakość naszej dokumentacji.



# Swagger UI



A screenshot of the Swagger UI interface. The header shows the Swagger logo and 'powered by SMARTBEAR'. The main title is 'Example API' with version '1.0' and 'OAS 3.0'. Below the title is a section for 'API description'. On the right, there is an 'Authorize' button with a lock icon. The main content area lists API endpoints under the 'user' resource. The endpoints are: POST /api/v1/user (Create a new user), GET /api/v1/user (Retrieve a list of users), GET /api/v1/user/{id} (Retrieve a user by ID), PATCH /api/v1/user/{id} (Update a user by ID), and DELETE /api/v1/user/{id} (Delete a user by ID). Each endpoint is color-coded: green for POST, blue for GET, teal for PATCH, and red for DELETE. Expandable arrows are visible on the right of each endpoint row.

Swagger  
powered by SMARTBEAR

## Example API 1.0 OAS 3.0

API description

Authorize 

### API

#### user

|        |                   |                          |   |
|--------|-------------------|--------------------------|---|
| POST   | /api/v1/user      | Create a new user        | ⌵ |
| GET    | /api/v1/user      | Retrieve a list of users | ⌵ |
| GET    | /api/v1/user/{id} | Retrieve a user by ID    | ⌵ |
| PATCH  | /api/v1/user/{id} | Update a user by ID      | ⌵ |
| DELETE | /api/v1/user/{id} | Delete a user by ID      | ⌵ |

# Przykładowo opisany endpoint



```
1  @Get()
2  @ApiOperation({
3      summary: "Get all users",
4      description: "Retrieve a list of all users in the system"
5  })
6  @ApiResponse({
7      status: 200,
8      description: "List of trips retrieved successfully",
9      type: [CreateUserDto]
10 })
11 findAll() {
12     return this.userService.findAll();
13 }
```



KN Solvro

# Część Praktyczna



KN Solvro

# Generowanie Zasobu



```
C:\Users\kstol\example>npx nest
```

```
Usage: nest <command> [options]
```

## Options:

```
-v, --version
```

Output the current version.

```
-h, --help
```

Output usage information.

## Commands:

```
new|n [options] [name]
```

Generate Nest application.

```
build [options] [apps...]
```

Build Nest application.

```
start [options] [app]
```

Run Nest application.

```
info|i
```

Display Nest project details.

```
add [options] <library>
```

Adds support for an external library to your project.

```
generate|g [options] <schematic> [name] [path]
```

Generate a Nest element.

Schematics available on @nestjs/schematics collection:

| name          | alias       | description                                  |
|---------------|-------------|----------------------------------------------|
| application   | application | Generate a new application workspace         |
| class         | cl          | Generate a new class                         |
| configuration | config      | Generate a CLI configuration file            |
| controller    | co          | Generate a controller declaration            |
| decorator     | d           | Generate a custom decorator                  |
| filter        | f           | Generate a filter declaration                |
| gateway       | ga          | Generate a gateway declaration               |
| guard         | gu          | Generate a guard declaration                 |
| interceptor   | itc         | Generate an interceptor declaration          |
| interface     | itf         | Generate an interface                        |
| library       | lib         | Generate a new library within a monorepo     |
| middleware    | mi          | Generate a middleware declaration            |
| module        | mo          | Generate a module declaration                |
| pipe          | pi          | Generate a pipe declaration                  |
| provider      | pr          | Generate a provider declaration              |
| resolver      | r           | Generate a GraphQL resolver declaration      |
| resource      | res         | Generate a new CRUD resource                 |
| service       | s           | Generate a service declaration               |
| sub-app       | app         | Generate a new application within a monorepo |



```
C:\Users\ksto1\example>npx nest g res user
```

```
? What transport layer do you use?
```

```
> REST API
```

```
GraphQL (code first)
```

```
GraphQL (schema first)
```

```
Microservice (non-HTTP)
```

```
WebSockets
```





```
C:\Users\kstol\example>npx nest g res user
✓ What transport layer do you use? REST API
? Would you like to generate CRUD entry points? (Y/n)
```





```
C:\Users\ksto1\example>npx nest g res user
✓ What transport layer do you use? REST API
✓ Would you like to generate CRUD entry points? Yes
CREATE src/user/user.controller.ts (917 bytes)
CREATE src/user/user.controller.spec.ts (576 bytes)
CREATE src/user/user.module.ts (250 bytes)
CREATE src/user/user.service.ts (633 bytes)
CREATE src/user/user.service.spec.ts (464 bytes)
CREATE src/user/dto/create-user.dto.ts (31 bytes)
CREATE src/user/dto/update-user.dto.ts (173 bytes)
CREATE src/user/entities/user.entity.ts (22 bytes)
UPDATE package.json (2082 bytes)
UPDATE src/app.module.ts (318 bytes)
- Installing packages (npm)...(node:12304) [DEP0190] DeprecationWarning: Passing args to a child process with shell option true can lead to security vulnerabilities, as the arguments are not escaped, only concatenated.
(Use 'node --trace-deprecation ...' to show where the warning was created)
✓ Packages installed successfully.
```



KN Solvro

Zawartość plików taka jak w rozdziale z przykładowymi

```
✓ user
  > dto
  > entities
  TS user.controller.spec.ts
  TS user.controller.ts
  TS user.module.ts
  TS user.service.spec.ts
  TS user.service.ts
```



KN Solvro

# Dodanie dokumentacji



```
C:\Users\kstol\example>npm i @nestjs/swagger  
added 5 packages, and audited 701 packages in 9s  
  
152 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```



KN Solvro

# Podłączenie serwisu bazy danych



## Moduł bazy danych

```
1 import { Module } from '@nestjs/common';
2 import { DatabaseService } from '../database.service';
3
4 @Module({
5   providers: [DatabaseService],
6   exports: [DatabaseService],
7 })
8 export class DatabaseModule {}
9
```

## Moduł użytkownika

```
1 import { Module } from '@nestjs/common';
2 import { UserService } from '../user.service';
3 import { UserController } from '../user.controller';
4 import { DatabaseModule } from 'src/database/database.module';
5
6 @Module({
7   controllers: [UserController],
8   providers: [UserService],
9   imports: [DatabaseModule],
10 })
11 export class UserModule {}
12
```

```
1 @Injectable()
2 export class UserService {
3   constructor(private databaseService: DatabaseService) {}
4 }
```



# Przykładowy skończony serwis

Zawiera obsługę wszystkich podstawowych operacji CRUD

```
1  @Injectable()
2  export class UserService {
3    constructor(private databaseService: DatabaseService) {}
4
5    async create(createUserDto: CreateUserDto): Promise<User> {
6      return this.databaseService.user.create({
7        data: {
8          name: createUserDto.name,
9          email: createUserDto.email,
10         age: createUserDto.age
11       }
12     });
13   }
14
15   async findAll(): Promise<User[]> {
16     return this.databaseService.user.findMany();
17   }
18
19   async findOne(id: number): Promise<User> {
20     const user = await this.databaseService.user.findUnique({
21       where: { id }
22     });
23     if(!user) {
24       throw new NotFoundException(`User with id ${id} not found`);
25     }
26     return user
27   }
28
29   async update(id: number, updateUserDto: UpdateUserDto): Promise<User> {
30     await this.findOne(id);
31     return this.databaseService.user.update({
32       where: { id },
33       data: {
34         name: updateUserDto.name,
35         email: updateUserDto.email,
36         age: updateUserDto.age
37       }
38     });
39   }
40
41   async remove(id: number): Promise<User> {
42     await this.findOne(id);
43     return this.databaseService.user.delete({
44       where: { id }
45     });
46   }
47 }
```

# Przykładowy skończony kontroler



```
1 @Controller('user')
2 export class UserController {
3     constructor(private readonly userService: UserService) {}
4
5     @Post()
6     async create(@Body() createUserDto: CreateUserDto) {
7         return this.userService.create(createUserDto);
8     }
9
10    @Get()
11    async findAll() {
12        return this.userService.findAll();
13    }
14
15    @Get('/:id')
16    async findOne(@Param('id', ParseIntPipe) id: number) {
17        return this.userService.findOne(id);
18    }
19
20    @Patch('/:id')
21    async update(@Param('id', ParseIntPipe) id: number, @Body() updateUserDto: UpdateUserDto) {
22        return this.userService.update(id, updateUserDto);
23    }
24
25    @Delete('/:id')
26    async remove(@Param('id', ParseIntPipe) id: number) {
27        return this.userService.remove(id);
28    }
29 }
```



# Walidacja danych - DTO



```
1  export class CreateUserDto {  
2      @IsString({ message: "Name must be a string" })  
3      @NotEmpty({ message: "Name is required" })  
4      @MaxLength(50, { message: "Name must be at most 50 characters" })  
5      name!: string;  
6  
7      @IsEmail()  
8  
9      email!: string;  
10  
11     @IsNumber({}, { message: "Age must be a number" })  
12     @Min(0, { message: "Age must be a positive number" })  
13     @Max(150, { message: "Age cannot be greater than 150" })  
14     age?: number;  
15 }  
16
```



# Dodawanie dokumentacji – main.ts

```
1  async function bootstrap() {
2    const app = await NestFactory.create(AppModule);
3
4    app.useGlobalPipes(new ValidationPipe({
5      whitelist: true,
6      transform: true,
7    }));
8    app.setGlobalPrefix('api/v1');
9
10   const config = new DocumentBuilder()
11     .setTitle("Example API")
12     .setDescription("API description")
13     .setVersion("1.0")
14     .addTag("API")
15     .addBearerAuth()
16     .build()
17
18   const documentFactory = () => SwaggerModule.createDocument(app, config);
19   SwaggerModule.setup("api", app, documentFactory)
20
21   await app.listen(process.env.PORT ?? 3000);
22 }
```



KN Solvro

# Dekoratory - kontroler

```
1 @Controller('user')
2 @ApiTags('user')
3 export class UserController {
4     constructor(private readonly userService: UserService) {}
5
6     @Post()
7     @ApiOperation({
8         summary: 'Create a new user',
9         description: 'Add a user to the database.'
10    })
11     @ApiResponse({
12         status: 201,
13         description: 'The user has been successfully created.',
14         type: CreateUserDtoResponse,
15     })
16     async create(@Body() createUserDto: CreateUserDto) {
17         return this.userService.create(createUserDto);
18     }
19
20     @Get()
21     @ApiOperation({
22         summary: 'Retrieve a list of users',
23         description: 'Retrieves a list of users from the database.'
24    })
25     @ApiResponse({
26         status: 200,
27         description: 'A list of users has been successfully retrieved.',
28         type: [CreateUserDtoResponse],
29     })
30     async findAll() {
31         return this.userService.findAll();
32     }
33 }
```

```
1 @Get('/:id')
2 @ApiOperation({
3     summary: 'Retrieve a user by ID',
4     description: 'Retrieves a user by their unique ID from the database.'
5 })
6 @ApiResponse({
7     status: 200,
8     description: 'The user has been successfully retrieved.',
9     type: CreateUserDtoResponse,
10 })
11 async findOne(@Param('id', ParseIntPipe) id: number) {
12     return this.userService.findOne(id);
13 }
14
15
16 @Patch('/:id')
17 @ApiOperation({
18     summary: 'Update a user by ID',
19     description: 'Updates a user by their unique ID from the database.'
20 })
21 @ApiResponse({
22     status: 200,
23     description: 'The user has been successfully updated.',
24     type: UpdateUserDto,
25 })
26 async update(@Param('id', ParseIntPipe) id: number, @Body() updateUserDto: UpdateUserDto) {
27     return this.userService.update(id, updateUserDto);
28 }
29
30 @Delete('/:id')
31 @ApiOperation({
32     summary: 'Delete a user by ID',
33     description: 'Deletes a user by their unique ID from the database.'
34 })
35 @ApiResponse({
36     status: 204,
37     description: 'The user has been successfully deleted.',
38 })
39 async remove(@Param('id', ParseIntPipe) id: number) {
40     return this.userService.remove(id);
41 }
42 }
```



KN Solvro

# Dekoratory dokumentacji - DTO



```
1 export class CreateUserDto {
2   @IsString({ message: "Name must be a string" })
3   @NotEmpty({ message: "Name is required" })
4   @MaxLength(50, { message: "Name must be at most 50 characters" })
5   @ApiProperty({
6     description: 'The name of the user',
7     example: 'John Doe',
8   })
9   name!: string;
10
11   @IsEmail()
12   @ApiProperty({
13     description: 'The email of the user',
14     example: 'johndoe@example.com'
15   })
16   email!: string;
17
18   @IsNumber({}, { message: "Age must be a number" })
19   @Min(0, { message: "Age must be a positive number" })
20   @Max(150, { message: "Age cannot be greater than 150" })
21   @ApiPropertyOptional({
22     description: 'The age of the user',
23     example: 30
24   })
25   age?: number;
26 }
27
```



# Paginacja - przykład

```
1 @ApiQuery({
2   name: 'offset', type: Number, required: false, description: "Offset param"
3 })
4 @ApiQuery({
5   name: 'limit', type: Number, required: false, description: 'Limit param'
6 })
7 async findAll(@Query('offset', ParseIntPipe) offset: number, @Query('limit', ParseIntPipe) limit: number) {
8   return this.userService.findAll(offset, limit);
9 }
```

```
1 async findAll(offset: number, limit: number): Promise<User[]> {
2   return this.databaseService.user.findMany({
3     skip: offset,
4     take: limit
5   });
6 }
```



KN Solvro

# DTO odpowiedzi (dla FE)



```
1 import { ApiProperty, ApiPropertyOptional } from "@nestjs/swagger";
2
3 export class CreateUserDtoResponse {
4   @ApiProperty({ description: 'The ID of the user', example: 1 })
5   id!: number;
6
7   @ApiProperty({ description: 'The name of the user', example: 'John Doe' })
8   name!: string;
9
10  @ApiProperty({ description: 'The email of the user', example: 'johndoe@example.com' })
11  email!: string;
12
13  @ApiPropertyOptional({ description: 'The age of the user', example: 30 })
14  age?: number;
15 }
```



## Example API 1.0 OAS 3.0

API description

Authorize 

### API

#### user

- POST** `/api/v1/user` Create a new user
- GET** `/api/v1/user` Retrieves a list of users
- GET** `/api/v1/user/{id}` Retrieves a user by ID
- PATCH** `/api/v1/user/{id}` Update a user by ID
- DELETE** `/api/v1/user/{id}` Delete a user by ID

### user

**POST** `/api/v1/user` Create a new user

Add a user to the database.

#### Parameters

Try it out

No parameters

#### Request body

application/json

Example Value | Schema

```
{
  "name": "John Doe",
  "email": "johndoe@example.com",
  "age": 30
}
```

#### Responses

| Code | Description                             |
|------|-----------------------------------------|
| 201  | The user has been successfully created. |

Links

No links

Media type

application/json

Controls Accept header

Example Value | Schema

```
{
  "id": 1,
  "name": "John Doe",
  "email": "johndoe@example.com",
  "age": 30
}
```



KN Solvro

# Homework



- 1. Przygotować CRUD dla uczestników (Participant), wydatków (Expense) i wycieczek (Trip)
- 2. Stworzyć DTO dla CRUDów, następnie zaimplementować walidację danych
- 3. Dodać i skonfigurować Swagger UI, następnie opisać endpointy i DTO w celu ich dokumentacji
- 4. Zapoznać się ze interfejsem Swagger UI i przetestować działanie endpointów
- **Zadanie Dodatkowe** - Zaimplementować paginację w wybranej przez siebie formie.



# Pytania?



KN Solvro

**Dziękuję za uwagę :)**



KN Solvro